```python
def normalize_cost(cost, max_cost):
    # Normalize cost between 0 and 1
    return round(cost / max_cost, 4)  # Limit to 4 decimal places

def calculate_cost_matrix(mission_matrix, current_state, battery_level):
    rows = len(mission_matrix)
    cols = len(mission_matrix[0])
    cost_matrix = [[0] * cols for _ in range(rows)]

    current_x, current_y = current_state
    max_cost = 2 * (rows + cols)  # Maximum possible cost

    for i in range(rows):
        for j in range(cols):
            if mission_matrix[i][j] == 1:
                # Calculate cost based on Manhattan distance from current state to
mission location
                cost = abs(i - current_y) + abs(j - current_x)
                normalized_cost = normalize_cost(cost, max_cost)
                # Adjust normalized cost based on battery level
                adjusted_cost = normalized_cost + (1 - normalized_cost) * (1 -
battery_level / 100)
                # Limit the number of decimal places
                adjusted_cost = round(adjusted_cost, 4)
                cost_matrix[i][j] = adjusted_cost
            else:
                # Set a high cost for locations where no mission is assigned
                cost_matrix[i][j] = float('inf')

    return cost_matrix

# Example mission matrix
mission_matrix = [
    [0, 0, 0, 0, 1],
    [0, 0, 1, 0, 0],
    [0, 1, 0, 0, 0],
    [0, 0, 0, 0, 1]
]

# Example current state and battery level
current_state = (2, 3)  # Assuming the drone's current location is at (2, 3)
battery_level = 70      # Assuming the drone's battery level is 70%

# Calculate cost matrix with battery level
cost_matrix = calculate_cost_matrix(mission_matrix, current_state, battery_level)

# Print cost matrix
print("Cost Matrix:")
for row in cost_matrix:
    print(row)
```